

EVIDENCIA DE DESEMPEÑO

GA7-220501096-AA3-EV02

Módulos de software codificados y probados

Proyecto: SIGEPOR / SICE

Sistema web para autenticación y gestión de usuarios, con persistencia en MySQL.

Tecnologías: Node.js, Express, React, Vite, MySQL, JWT y bcrypt.

Fecha de entrega: 18 de agosto de 2026

Propósito de la evidencia

Documentar la codificación, integración, persistencia, validación funcional y control de versiones del módulo desarrollado.

1. Módulos desarrollados e integración

La solución se organizó por responsabilidades para facilitar mantenimiento y evolución. El backend expone una API REST y el frontend consume sus servicios mediante Axios.

Backend modular

- `src/app.js`: Express y rutas.
- `src/server.js`: arranque y conexión.
- `src/modules/auth`: login JWT.
- `src/modules/users`: CRUD y estado.
- Repositorios, servicios, validadores y middleware.

Frontend web

- `sice-frontend/src`: aplicación React.
- Login y sesión persistente.
- Dashboard y perfil.
- Crear, editar, consultar y eliminar usuarios.
- Rutas protegidas y por rol.



Programación orientada a objetos y buenas prácticas

Aunque el código usa módulos funcionales de Node.js en lugar de clases en todos los componentes, aplica encapsulamiento por capas, separación de responsabilidades, reutilización de servicios y manejo centralizado de errores. Las credenciales se obtienen desde variables de entorno.

Repositorio

<https://github.com/julioesanchez62-web/sigepor-backend>

Nota: la URL corresponde al repositorio reportado para la entrega. El remoto debe estar configurado localmente antes del envío final.

2. Persistencia y funcionalidades implementadas

Persistencia MySQL

- Pool de conexiones con `mysql2/promise`.
- Tablas `usuario` y `rol`.
- Relación usuario-rol mediante clave foránea.
- Eliminación lógica mediante `deleted_at`.
- Validación de conexión con `SELECT 1`.

Seguridad

- Contraseñas cifradas con `bcrypt`.
- Tokens JWT con expiración configurable.
- Middleware para proteger recursos.
- Validación de datos de entrada.
- Mensajes de autenticación no reveladores.

Módulo	Operaciones verificables	Resultado esperado
Autenticación	Login y perfil autenticado	Token JWT y datos del usuario
Usuarios	Crear, listar, consultar, actualizar	Respuesta JSON y persistencia MySQL
Estado	Activar, desactivar y eliminar lógicamente	El registro permanece trazable
Frontend	Login, dashboard, perfil y CRUD	Flujo integrado con la API

Captura sugerida: estructura del proyecto

Insertar aquí una captura del explorador de VS Code mostrando

```
src/  
  ,  
sice-frontend/src/  
  ,  
  database/  
y los archivos de configuración.
```

Captura sugerida: funcionalidades

Insertar aquí una captura del login o del CRUD de usuarios funcionando en el navegador.

3. Pruebas funcionales y base de datos

La validación se realiza con el backend configurado y MySQL disponible. Los comandos reproducibles son:

```
npm install
npm run dev
GET http://localhost:3001/
cd sice-frontend
npm run build
npm run lint
```

Prueba	Validación	Evidencia
Salud de API	GET / responde JSON	Respuesta HTTP 200
Autenticación	Credenciales válidas e inválidas	JWT o error 401
CRUD usuarios	Crear, consultar, actualizar y eliminar lógico	Datos reflejados en MySQL
Frontend	Build y lint de Vite	Compilación sin errores

Captura: base de datos

Insertar captura de MySQL Workbench con las
tablas
usuario
y
rol
, o de una consulta
SHOW TABLES
.

Captura: resultados

Insertar captura de Postman, navegador o
terminal con respuestas exitosas y códigos HTTP.

Transparencia: el proyecto todavía no contiene una suite automatizada; el script raíz `npm test` está pendiente. Las pruebas de esta entrega deben respaldarse con capturas de ejecución manual y con los comandos de build/lint del frontend.

4. Control de versiones, README y conclusiones

Historial Git

El repositorio local conserva commits incrementales para la implementación del módulo de usuarios:

- 8a7f4f4 chore: initial project commit
- d4e7793 soft delete y actualización de estado
- 36bbe32 eliminación lógica
- f03dc75 creación de usuarios por capas
- cb61c3a inicialización Node.js

Captura: historial

Insertar captura de
`git log --oneline`
o del historial del repositorio remoto.

README actualizado

El README describe instalación, variables de entorno, ejecución del backend, frontend React, endpoints, validación y limitaciones conocidas. El archivo entregable es `README.md`.

Conclusiones

- Se integraron módulos de autenticación y usuarios con una arquitectura backend por capas.
- La persistencia en MySQL y el pool de conexiones permiten conservar información fuera del navegador.
- React y las rutas protegidas completan el flujo web de usuario.
- Git documenta la evolución del desarrollo y facilita la trazabilidad.
- Como mejora inmediata se debe agregar una suite automatizada y publicar/verificar el remoto antes de entregar.

Checklist de entrega

- ☐ Captura estructura ☐ Captura funcionalidades ☐ Captura BD ☐ Captura pruebas ☐ Captura commits
☐ README actualizado